

API & Validation

A practical guide to Fetch API, Axios, Postman, GraphQL, Zod and Swagger

Structured as a learning path, following the API and backend tracks on **roadmap.sh**.

Topic	What you will learn
Fetch API	Built-in browser/Node HTTP client, error handling, timeouts
Axios	Instances, interceptors, cleaner errors
Postman	Collections, environments, automated tests
GraphQL	Queries, mutations, schema, calling from JavaScript
Zod	Schema validation for requests, responses and forms
Swagger / OpenAPI	Documenting and exploring your API

Reference: <https://roadmap.sh/> (Backend and API Design roadmaps)

1. Learning path

roadmap.sh describes API design as creating interfaces that expose data and functionality for other software to use, and points out that you should understand the main API styles (REST, SOAP and GraphQL) along with standards, best practices and security. Its backend roadmap suggests learning to build a simple RESTful API and add basic authentication and authorization. The order below follows that idea, from consuming APIs to documenting them.

Step	Topic	Goal
1	HTTP basics	Methods, status codes, headers, JSON
2	Fetch API	Call an API from JavaScript with no dependencies
3	Axios	A friendlier client with interceptors
4	Postman	Explore and test endpoints before writing code
5	GraphQL	Ask for exactly the fields you need
6	Zod	Validate anything that crosses a trust boundary
7	Swagger / OpenAPI	Describe the API so others can use it
8	Security	Auth, rate limiting, safe error handling

HTTP essentials

Method	Purpose	Typical success code
GET	Read a resource (no body, safe to repeat)	200 OK
POST	Create a resource	201 Created
PUT	Replace a resource entirely	200 OK / 204 No Content
PATCH	Update part of a resource	200 OK
DELETE	Remove a resource	204 No Content

Range	Meaning	Common examples
2xx	Success	200, 201, 204
3xx	Redirect	301, 304
4xx	Client error	400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 422 validation failed, 429 too many requests
5xx	Server error	500, 502, 503

2. Fetch API

fetch() is built into modern browsers and Node.js (18+). It returns a Promise that resolves to a **Response**. The most common mistake: **fetch does not reject on HTTP errors** such as 404 or 500. It only rejects on network failures, so always check `response.ok`.

GET request

```
async function getUsers() {
  const res = await fetch("https://api.example.com/users");

  if (!res.ok) {
    throw new Error(`HTTP ${res.status}: ${res.statusText}`);
  }
  return res.json();          // parses the JSON body
}

getUsers().then(console.log).catch(console.error);
```

POST request with JSON

```
async function createUser(data) {
  const res = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(data),    // you must stringify yourself
  });

  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

Timeout and cancellation

Use an **AbortController**. `AbortSignal.timeout()` is a shortcut in modern runtimes.

```
// Shortcut: abort automatically after 5 seconds
const res = await fetch(url, { signal: AbortSignal.timeout(5000) });

// Manual control (e.g. cancel when a component unmounts)
const controller = new AbortController();
fetch(url, { signal: controller.signal });
controller.abort();
```

A reusable wrapper

```
async function api(path, options = {}) {
  const res = await fetch(`https://api.example.com${path}`, {
    ...options,
    headers: { "Content-Type": "application/json", ...options.headers },
  });

  const data = res.status === 204 ? null : await res.json();

  if (!res.ok) {
```

```
    const err = new Error(data?.message ?? `HTTP ${res.status}`);
    err.status = res.status;
    err.body = data;
    throw err;
  }
  return data;
}
```

3. Axios

Axios is a promise-based HTTP client for browsers and Node.js. Compared with fetch it parses JSON automatically, rejects on non-2xx responses, supports interceptors and makes shared configuration easy.

```
npm install axios
```

Basic requests

```
import axios from "axios";

// GET with query params -> /users?page=2&limit=10
const { data } = await axios.get("https://api.example.com/users", {
  params: { page: 2, limit: 10 },
});

// POST: object is serialized to JSON automatically
const created = await axios.post("https://api.example.com/users", {
  name: "Sara",
  email: "sara@example.com",
});

// PUT / PATCH / DELETE follow the same shape
await axios.patch("/users/42", { name: "Sara K." });
await axios.delete("/users/42");
```

Instance with defaults and interceptors

```
const client = axios.create({
  baseURL: "https://api.example.com",
  timeout: 8000,
  headers: { "Content-Type": "application/json" },
});

// Attach the auth token to every request
client.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Handle errors in one place
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // e.g. refresh the token or redirect to login
    }
    return Promise.reject(error);
  }
);
```

Error handling

```
try {
  await client.get("/users/999");
} catch (error) {
  if (axios.isAxiosError(error)) {
    if (error.response) {
```

```
    // Server answered with a non-2xx status
    console.log(error.response.status, error.response.data);
  } else if (error.request) {
    // Request sent, no response (network down, timeout)
    console.log("No response received");
  }
} else {
  throw error;
}
}
```

Fetch vs Axios

Feature	Fetch	Axios
Installation	Built in	npm package
JSON parsing	Manual: <code>res.json()</code>	Automatic: <code>response.data</code>
HTTP errors (4xx/5xx)	Does not reject; check <code>res.ok</code>	Rejects by default
Interceptors	Write your own wrapper	Built in
Timeout	<code>AbortSignal.timeout()</code>	<code>timeout</code> option
Upload progress	Limited	<code>onUploadProgress</code>
Bundle size	None	Small dependency

4. Postman

Postman is a tool for building, sending and testing API requests without writing client code. Use it to explore an API, share requests with your team and run automated checks.

Core concepts

Concept	What it is
Request	One HTTP call: method, URL, params, headers, body, auth
Collection	A folder of related requests you can share, document and run together
Environment	A set of variables (dev, staging, prod) such as <code>{{baseUrl}}</code> and <code>{{token}}</code>
Variables	Reusable values in <code>{{double braces}}</code> ; scopes: global, collection, environment, local
Pre-request script	JavaScript that runs before a request (set a timestamp, sign a request)
Tests	JavaScript that runs after the response and asserts things about it
Collection Runner	Runs every request in a collection in order, with test results

Typical workflow

- Create an **Environment** with `baseUrl` and `token`.
- Create a **Collection** and add requests using `{{baseUrl}}/users`.
- Under **Authorization**, choose Bearer Token and use `{{token}}`.
- Write **Tests** to assert status, timing and body shape.
- Run the collection with the **Collection Runner**, or from CI using the Postman CLI or Newman.

Example test script (Tests tab)

```
pm.test("Status is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Responds in under 500 ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test("Returns an array of users with an email", function () {
  const users = pm.response.json();
  pm.expect(users).to.be.an("array");
  pm.expect(users[0]).to.have.property("email");
});

// Save a value for the next request (chaining)
const body = pm.response.json();
pm.environment.set("userId", body[0].id);
```

Tip: Postman can import an OpenAPI (Swagger) file and generate a whole collection from it, which keeps your docs and your tests in sync.

5. GraphQL

GraphQL is a query language for APIs. Instead of many endpoints, there is usually one endpoint (often `/graphql`), and the client describes exactly which fields it wants. That avoids over-fetching and under-fetching.

	REST	GraphQL
Endpoints	Many (<code>/users</code> , <code>/users/1/posts</code>)	Usually one (<code>/graphql</code>)
Response shape	Fixed by the server	Chosen by the client
Related data	Several requests	One nested query
Typing	Optional (via OpenAPI)	Built-in strongly typed schema
Caching	Simple HTTP caching	Needs client-side or persisted-query caching

Schema (SDL)

```
type User {
  id: ID!
  name: String!
  email: String!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  author: User!
}

type Query {
  user(id: ID!): User
  users(limit: Int = 10): [User!]!
}

type Mutation {
  createUser(name: String!, email: String!): User!
}
```

Query and mutation

```
# Query: only the fields you list are returned
query GetUser($id: ID!) {
  user(id: $id) {
    name
    posts {
      title
    }
  }
}

# Mutation: change data, and ask for fields back
mutation CreateUser($name: String!, $email: String!) {
  createUser(name: $name, email: $email) {

```

```
    id
    name
  }
}
```

Calling GraphQL from JavaScript

A GraphQL call is just an HTTP POST with a query and variables. Note that GraphQL usually returns HTTP 200 even when something fails, so check the errors field.

```
async function gql(query, variables) {
  const res = await fetch("https://api.example.com/graphql", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ query, variables }),
  });

  const { data, errors } = await res.json();
  if (errors?.length) throw new Error(errors[0].message);
  return data;
}

const data = await gql(
  `query ($id: ID!) { user(id: $id) { name email } }`,
  { id: "42" }
);
```

The same call with Axios

```
const { data } = await axios.post("https://api.example.com/graphql", {
  query: `query ($id: ID!) { user(id: $id) { name email } }`,
  variables: { id: "42" },
});
if (data.errors) throw new Error(data.errors[0].message);
console.log(data.data.user);
```

Popular tools: Apollo Server / Apollo Client, GraphQL Yoga, urql, graphql-request, and GraphiQL or Apollo Sandbox for exploring a schema. In Postman you can create a GraphQL request type and load the schema by introspection.

6. Validation with Zod

TypeScript types disappear at runtime, so they cannot protect you from bad data arriving in a request body, a query string, or a third-party API response. **Zod** is a TypeScript-first schema library: you define a schema once, and get both runtime validation and a static type.

```
npm install zod
```

Defining a schema

```
import { z } from "zod";

const CreateUserSchema = z.object({
  name: z.string().min(2, "Name must be at least 2 characters"),
  email: z.string().email("Invalid email"),
  age: z.number().int().min(18).optional(),
  role: z.enum(["admin", "user"]).default("user"),
  tags: z.array(z.string()).max(5).default([]),
});

// Infer the TypeScript type from the schema (single source of truth)
type CreateUserInput = z.infer<typeof CreateUserSchema>;
```

parse vs safeParse

```
// parse: returns the data or THROWS a ZodError
const user = CreateUserSchema.parse(input);

// safeParse: never throws, returns a result object
const result = CreateUserSchema.safeParse(input);

if (!result.success) {
  console.log(result.error.issues);
  // [{ path: ["email"], message: "Invalid email", code: "invalid_string", ... }]
} else {
  console.log(result.data);    // fully typed and validated
}
```

Version note: the API shown here (`z.object`, `z.string().email()`, `safeParse`, `z.infer`) works across recent Zod releases. Newer major versions add some top-level helpers (for example `z.email()`) and change parts of error formatting. Check the Zod docs for the version you install.

Validating an Express request body (middleware)

```
import express from "express";
import { z } from "zod";

const app = express();
app.use(express.json());

const validate = (schema) => (req, res, next) => {
  const result = schema.safeParse(req.body);
  if (!result.success) {
    return res.status(422).json({
      message: "Validation failed",
      errors: result.error.issues.map((i) => ({
```

```

        field: i.path.join("."),
        message: i.message,
      })),
    });
  }
  req.body = result.data;      // use the cleaned, typed data
  next();
};

app.post("/users", validate(CreateUserSchema), (req, res) => {
  // req.body is guaranteed to match the schema here
  res.status(201).json({ id: 1, ...req.body });
});

```

Validating an API response (Fetch + Zod)

Never trust data from outside your code, even from an API you call. Parse it at the boundary.

```

const UserSchema = z.object({
  id: z.number(),
  name: z.string(),
  email: z.string().email(),
});
const UsersSchema = z.array(UserSchema);

async function getUsers() {
  const res = await fetch("https://api.example.com/users");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);

  const json = await res.json();
  return UsersSchema.parse(json); // throws if the API changed shape
}

```

Useful features

```

// Query strings arrive as strings: coerce them
const QuerySchema = z.object({
  page: z.coerce.number().int().positive().default(1),
  limit: z.coerce.number().int().min(1).max(100).default(10),
});

// Cross-field rules
const SignupSchema = z
  .object({ password: z.string().min(8), confirm: z.string() })
  .refine((d) => d.password === d.confirm, {
    message: "Passwords do not match",
    path: ["confirm"],
  });

// Reuse and derive schemas
const UpdateUserSchema = CreateUserSchema.partial(); // all optional
const PublicUserSchema = CreateUserSchema.pick({ name: true, email: true });
const WithId = CreateUserSchema.extend({ id: z.number() });

// Transform after validating
const Trimmed = z.string().trim().toLowerCase();

```

Zod with React forms

```

// npm install react-hook-form @hookform/resolvers
import { useForm } from "react-hook-form";

```

```
import { zodResolver } from "@hookform/resolvers/zod";

const { register, handleSubmit, formState: { errors } } =
  useForm({ resolver: zodResolver(CreateUserSchema) });
```

Other validators you will see: Joi, Yup, Valibot, class-validator, AJV (JSON Schema).

7. Swagger / OpenAPI

OpenAPI is the standard format for describing REST APIs in YAML or JSON. **Swagger** is the family of tools around it, such as Swagger UI (interactive documentation you can try in the browser), Swagger Editor and Swagger Codegen. A good spec can generate docs, client SDKs, server stubs and Postman collections.

Example OpenAPI 3.0 spec (openapi.yaml)

```
openapi: 3.0.3
info:
  title: Users API
  version: 1.0.0
servers:
  - url: https://api.example.com
paths:
  /users:
    get:
      summary: List users
      parameters:
        - in: query
          name: limit
          schema: { type: integer, default: 10 }
      responses:
        "200":
          description: A list of users
          content:
            application/json:
              schema:
                type: array
                items: { $ref: "#/components/schemas/User" }
    post:
      summary: Create a user
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/CreateUser" }
      responses:
        "201":
          description: Created
          content:
            application/json:
              schema: { $ref: "#/components/schemas/User" }
        "422":
          description: Validation failed
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    User:
      type: object
      required: [id, name, email]
      properties:
        id: { type: integer }
```

```
    name: { type: string }
    email: { type: string, format: email }
  CreateUser:
    type: object
    required: [name, email]
    properties:
      name: { type: string, minLength: 2 }
      email: { type: string, format: email }
```

Serving Swagger UI from Express

```
npm install swagger-ui-express yaml
```

```
import fs from "node:fs";
import YAML from "yaml";
import swaggerUi from "swagger-ui-express";

const spec = YAML.parse(fs.readFileSync("./openapi.yaml", "utf8"));

app.use("/docs", swaggerUi.serve, swaggerUi.setup(spec));
// Open http://localhost:3000/docs
```

Keeping docs and validation in sync

Writing a Zod schema *and* an OpenAPI schema by hand means they drift apart. Two common ways to avoid that: generate the OpenAPI document from your Zod schemas (for example with the [@asteasolutions/zod-to-openapi](#) package), or go the other way and generate types and clients from the OpenAPI file. Pick one source of truth.

Approach	Source of truth	Good for
Code-first	Zod schemas / annotations in code	Teams that mainly work in TypeScript
Design-first	openapi.yaml written before code	Cross-team APIs, public APIs, contract review

8. Putting it together

A small end-to-end example: a shared Zod schema, an Express endpoint that validates input, and an Axios client that validates what comes back.

schemas.ts

```
import { z } from "zod";

export const CreateUserSchema = z.object({
  name: z.string().min(2),
  email: z.string().email(),
});
export const UserSchema = CreateUserSchema.extend({ id: z.number() });

export type CreateUser = z.infer<typeof CreateUserSchema>;
export type User = z.infer<typeof UserSchema>;
```

server.ts

```
import express from "express";
import { CreateUserSchema } from "../schemas";

const app = express();
app.use(express.json());

let nextId = 1;
app.post("/users", (req, res) => {
  const parsed = CreateUserSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(422).json({ errors: parsed.error.issues });
  }
  res.status(201).json({ id: nextId++, ...parsed.data });
});

app.listen(3000);
```

client.ts

```
import axios from "axios";
import { CreateUser, UserSchema } from "../schemas";

const client = axios.create({ baseURL: "http://localhost:3000" });

export async function createUser(input: CreateUser) {
  const { data } = await client.post("/users", input);
  return UserSchema.parse(data); // validate the response too
}
```

Test it in Postman

- POST `{{baseUr l}}/users` with a valid JSON body and expect **201**.
- Send a bad email or a one-character name and expect **422** with an errors array.
- Add the Tests script from section 4 to assert status and body shape.

9. Best practices checklist

roadmap.sh's API design material stresses that API security covers protecting data, preventing unauthorized access and keeping the hosting system safe. The checklist below turns that into concrete habits.

Design

- Use nouns for resources (`/users`), verbs come from the HTTP method.
- Return correct status codes and a consistent error shape.
- Paginate list endpoints and version the API (`/v1/...`).
- Document every endpoint in OpenAPI and keep it in version control.

Validation

- Validate on the server always. Client-side validation is only for user experience.
- Validate body, query string, path params and headers.
- Reject unknown or oversized input; set body size limits.
- Validate responses from third-party APIs before using them.
- Use 400 or 422 for invalid input and return field-level messages.

Security

- Use HTTPS everywhere; never send tokens in URLs.
- Authenticate (JWT, OAuth 2.0, API keys) and authorize every request.
- Add rate limiting and set CORS to specific origins, not a wildcard.
- Do not leak stack traces or internal details in error responses.
- Keep secrets in environment variables, never in the repository.

Client side

- Always set timeouts and handle network failures.
- Centralize base URL, auth and error handling (Axios instance or fetch wrapper).
- Retry only safe or idempotent requests, with backoff.

Resources

Topic	Where to look
Learning paths	roadmap.sh: Backend, API Design and API Security Best Practices pages
Fetch	MDN Web Docs: Fetch API
Axios	axios-http.com

Topic	Where to look
Postman	learning.postman.com
GraphQL	graphql.org/learn
Zod	zod.dev
OpenAPI / Swagger	swagger.io and spec.openapis.org